

Creation of Threads – Thread Class, Runnable Interface, Main Thread & Multithreading

Page 1 — Introduction to Threads

1. What is a Thread?

A **thread** is a lightweight unit of execution within a program.

A Java program can perform multiple tasks at the same time using multiple threads.

Example: A music application can:

- Play music
- Download a song
- Display animation
- Respond to user clicks

All these tasks can be handled using different threads.

Process vs Thread

Process	Thread
Independent program in execution	Small unit of a process
Requires more resources	Requires fewer resources
Has its own memory space	Shares memory with other threads
More expensive to create	Less expensive to create
Example: Chrome application	A task running inside Chrome

2. What is Multithreading?

Multithreading is the execution of multiple threads concurrently within a single program.

Java Program

```
|  
+---- Thread 1 → Task A  
|  
+---- Thread 2 → Task B  
|  
+---- Thread 3 → Task C
```

Advantages of Multithreading

- Better CPU utilization
- Faster execution of independent tasks
- Improved application responsiveness
- Allows multiple tasks simultaneously
- Useful for games, servers, animations and networking

Real-Life Examples

Web Browser

Browser

```
|  
├─ Downloading  
├─ Playing Video  
├─ Loading Web Page  
└─ User Interaction
```

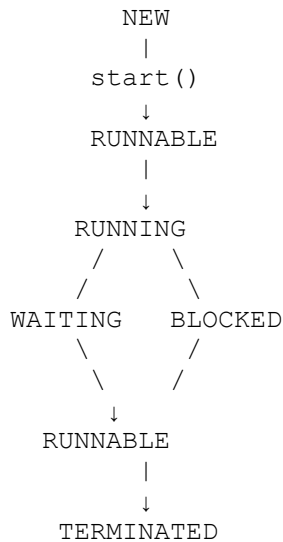
Online Banking

Banking Application

```
|  
├─ Transaction Processing  
├─ Notification  
├─ Database Operation  
└─ User Interface
```

Thread Life Cycle

A thread passes through different states during its execution.



Important Thread States

1. NEW

Thread object is created but `start()` has not been called.

```
Thread t = new Thread();
```

2. RUNNABLE

Thread is ready to run.

```
t.start();
```

3. RUNNING

The thread is actually executing its `run()` method.

4. WAITING / BLOCKED

Thread temporarily stops execution because it is waiting for a resource, notification or lock.

5. TERMINATED

Thread has completed execution.

`start()` VS `run()`

This is very important.

```
t.start();
```

`start()` creates a **new thread** and then JVM calls `run()`.

```
t.run();
```

Calling `run()` directly **does not create a new thread**. It behaves like a normal method call.

Example

```
class Demo extends Thread {  
    public void run() {  
        System.out.println("Thread is running");  
    }  
  
    public static void main(String[] args) {  
        Demo t = new Demo();  
        t.start();  
    }  
}
```

Output:

```
Thread is running
```

Creating Thread Using Thread Class

Java provides the `Thread` class for creating threads.

It belongs to:

```
java.lang.Thread
```

Since `java.lang` is automatically imported, no explicit import is required.

Steps

```
Create class
      ↓
Extend Thread
      ↓
Override run()
      ↓
Create object
      ↓
Call start()
```

Syntax

```
class MyThread extends Thread {

    public void run() {
        // thread task
    }
}
```

Create and start:

```
MyThread t = new MyThread();
t.start();
```

Example 1: Simple Thread

```
class MyThread extends Thread {

    public void run() {
        System.out.println("My thread is running");
    }

    public static void main(String[] args) {

        MyThread t = new MyThread();

        t.start();
    }
}
```

Output

My thread is running

Example 2: Printing Numbers

```
class NumberThread extends Thread {  
    public void run() {  
        for (int i = 1; i <= 5; i++) {  
            System.out.println(i);  
        }  
    }  
  
    public static void main(String[] args) {  
        NumberThread t = new NumberThread();  
        t.start();  
    }  
}
```

Possible output:

```
1  
2  
3  
4  
5
```

Important Point

The `run()` method contains the **task that the thread performs**.

```
public void run() {  
    // task of thread  
}
```

Thread Class Methods

The `Thread` class provides several useful methods.

Common Thread Methods

Method	Purpose
<code>start()</code>	Starts a new thread
<code>run()</code>	Contains thread task
<code>sleep()</code>	Pauses thread temporarily
<code>join()</code>	Waits for another thread to finish
<code>getName()</code>	Returns thread name
<code>setName()</code>	Sets thread name
<code>getPriority()</code>	Gets priority
<code>setPriority()</code>	Sets priority
<code>currentThread()</code>	Returns currently executing thread
<code>isAlive()</code>	Checks whether thread is alive

`sleep()`

`sleep()` pauses the current thread for a specified time.

```
Thread.sleep(1000);
```

1000 milliseconds = **1 second**

Example:

```
class Demo extends Thread {  
    public void run() {  
        for (int i = 1; i <= 5; i++) {  
            System.out.println(i);  
            try {  
                Thread.sleep(1000);  
            }  
            catch (InterruptedException e) {  
                System.out.println(e);  
            }  
        }  
    }  
}
```

```

        public static void main(String[] args) {

            Demo t = new Demo();
            t.start();

        }
    }

```

Output appears approximately one number per second.

getName()

```
System.out.println(Thread.currentThread().getName());
```

setName()

```
t.setName("MyThread");
```

join()

join() makes one thread wait until another thread completes.

```

t.start();
t.join();

```

Example:

```

class Demo extends Thread {

    public void run() {
        System.out.println("Child Thread");
    }

    public static void main(String[] args)
        throws InterruptedException {

        Demo t = new Demo();

        t.start();
        t.join();

        System.out.println("Main Thread");
    }
}

```

Possible output:

```

Child Thread
Main Thread

```

Creating Thread Using Runnable Interface

Another way to create a thread is by implementing the **Runnable interface**.

```
java.lang.Runnable
```

Runnable contains one important method:

```
public void run()
```

Steps

```
Create class
      ↓
Implement Runnable
      ↓
Override run()
      ↓
Create Runnable object
      ↓
Create Thread object
      ↓
Call start()
```

Syntax

```
class MyTask implements Runnable {

    public void run() {
        // task
    }
}
```

Then:

```
MyTask task = new MyTask();

Thread t = new Thread(task);

t.start();
```

Example

```
class MyTask implements Runnable {

    public void run() {

        System.out.println("Runnable thread is running");
    }

    public static void main(String[] args) {

        MyTask task = new MyTask();
```

```

        Thread t = new Thread(task);

        t.start();
    }
}

```

Output

Runnable thread is running

Why Runnable?

Java does not support multiple inheritance of classes.

Suppose:

```
class Student extends Person
```

Now if the same class needs to behave as a thread, it cannot do:

```
class Student extends Person, Thread
```

This is not allowed.

Instead:

```
class Student extends Person implements Runnable
```

Therefore, `Runnable` provides greater flexibility.

Thread Class vs Runnable

Thread	Runnable
Extend <code>Thread</code>	Implement <code>Runnable</code>
Cannot extend another class	Can extend another class
Simpler for basic examples	More flexible
Task and thread are closely coupled	Task can be separated from Thread

Main Thread

When a Java program starts, JVM automatically creates a thread called the **main thread**.

Execution starts from:

```
public static void main(String[] args)
```

Therefore, `main()` executes inside the main thread.

Example

```
class MainThreadDemo {  
    public static void main(String[] args) {  
        Thread t = Thread.currentThread();  
        System.out.println("Thread Name: " + t.getName());  
        System.out.println("Thread Priority: " + t.getPriority());  
    }  
}
```

Possible output:

```
Thread Name: main  
Thread Priority: 5
```

Thread.currentThread()

Returns the thread that is currently executing.

```
Thread t = Thread.currentThread();
```

Changing Main Thread Name

```
class Demo {  
    public static void main(String[] args) {  
        Thread t = Thread.currentThread();  
        System.out.println(t.getName());  
        t.setName("MainThread");  
        System.out.println(t.getName());  
    }  
}
```

Output:

```
main  
MainThread
```

Main Thread and Child Thread

```
class Demo extends Thread {  
  
    public void run() {  
        System.out.println("Child Thread");  
    }  
  
    public static void main(String[] args) {  
  
        System.out.println("Main Thread");  
  
        Demo t = new Demo();  
  
        t.start();  
    }  
}
```

Possible output:

```
Main Thread  
Child Thread
```

The exact order of thread output can vary because thread scheduling is controlled by the JVM/operating system.

Creating Multiple Threads

Multiple threads can be created in a single Java program.

Example

```
class MyThread extends Thread {  
    private String name;  
  
    MyThread(String name) {  
        this.name = name;  
    }  
  
    public void run() {  
        for (int i = 1; i <= 5; i++) {  
            System.out.println(name + " : " + i);  
  
            try {  
                Thread.sleep(500);  
            }  
            catch (InterruptedException e) {  
                System.out.println(e);  
            }  
        }  
    }  
  
    public static void main(String[] args) {  
        MyThread t1 = new MyThread("Thread 1");  
        MyThread t2 = new MyThread("Thread 2");  
        MyThread t3 = new MyThread("Thread 3");  
  
        t1.start();  
        t2.start();  
        t3.start();  
    }  
}
```

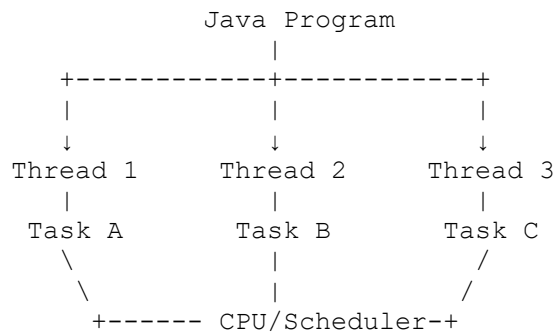
Possible Output

```
Thread 1 : 1  
Thread 2 : 1  
Thread 3 : 1  
Thread 2 : 2  
Thread 1 : 2  
Thread 3 : 2  
Thread 1 : 3  
...
```

The exact output order is **not guaranteed**.

Why?

The JVM's thread scheduler decides which runnable thread gets CPU time.



Important

Calling:

```
t1.start();
t2.start();
t3.start();
```

does **not** mean:

```
Thread 1 finishes
    ↓
Thread 2 finishes
    ↓
Thread 3 finishes
```

Instead, they may execute concurrently.

Multithreading Using Runnable + Summary

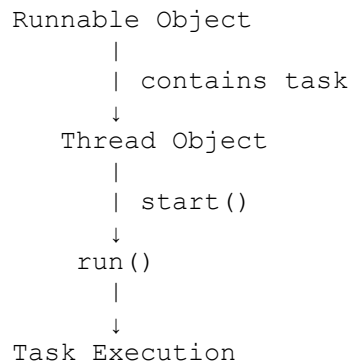
Multiple threads can also be created using the `Runnable` interface.

Example

```
class MyTask implements Runnable {  
    private String name;  
  
    MyTask(String name) {  
        this.name = name;  
    }  
  
    public void run() {  
        for (int i = 1; i <= 5; i++) {  
            System.out.println(name + " : " + i);  
  
            try {  
                Thread.sleep(500);  
            }  
            catch (InterruptedException e) {  
                System.out.println(e);  
            }  
        }  
    }  
  
    public static void main(String[] args) {  
        MyTask task1 = new MyTask("Task 1");  
        MyTask task2 = new MyTask("Task 2");  
  
        Thread t1 = new Thread(task1);  
        Thread t2 = new Thread(task2);  
  
        t1.start();  
        t2.start();  
    }  
}
```

Important Concept

`Runnable` represents the **task**, while `Thread` represents the **execution mechanism**.



Thread Class vs Runnable Interface — Quick Revision

Feature	Thread	Runnable
Type	Class	Interface
Syntax	<code>extends Thread</code>	<code>implements Runnable</code>
Task	<code>run()</code>	<code>run()</code>
Start	<code>start()</code>	<code>new Thread(obj).start()</code>
Can extend another class?	✗ No	☑ Yes
Suitable for	Simple thread programs	Flexible/design-oriented programs

Thread Class vs Runnable Interface

Basis	Thread Class	Runnable Interface
Type	Class	Functional interface
Package	<code>java.lang</code>	<code>java.lang</code>
Implementation	Extend <code>Thread</code>	Implement <code>Runnable</code>
Syntax	<code>class MyThread extends Thread</code>	<code>class MyTask implements Runnable</code>
Main method to override	<code>run()</code>	<code>run()</code>
How to start	Call <code>object.start()</code>	Create <code>Thread</code> object and call <code>start()</code>
Thread object	The class object itself is a thread	<code>Runnable</code> object represents the task; <code>Thread</code> executes it
Inheritance	Cannot extend another class	Can extend another class
Multiple inheritance flexibility	Less flexible	More flexible
Task & thread relationship	Task and thread are combined	Task and thread are separated
Code reusability	Comparatively lower	Comparatively higher
Sharing same task	Less convenient	Multiple <code>Thread</code> objects can use the same <code>Runnable</code> object
Memory/resource perspective	Each subclass object represents a thread	<code>Runnable</code> represents a task; threads can be created separately
Suitable for	Simple thread-based programs	Applications where task and execution should be separated
Example	<code>class A extends Thread</code>	<code>class A implements Runnable</code>
Java inheritance limitation	Uses class inheritance	Leaves class inheritance available
Recommended when	You specifically need to customize/extend <code>Thread</code>	You only need to define a task to be executed by a thread

Basis	Thread Class	Runnable Interface
Main Features	Simple thread creation, <code>run()</code> method, <code>start()</code> , thread methods like <code>sleep()</code> , <code>join()</code> , <code>getName()</code> , and <code>setPriority()</code>	Task-based design, reusable task, supports class inheritance, separates task from thread, works with <code>Thread</code> and <code>start()</code>